Rapport écrit par Emelyne LEFEVRE

Projet 1 :
Création d'une base de données pour le projet « Création d'entreprise de vente de vêtements de grande marque »

Introduction :

Lys & Or, une marque prestigieuse de prêt-à-porter.

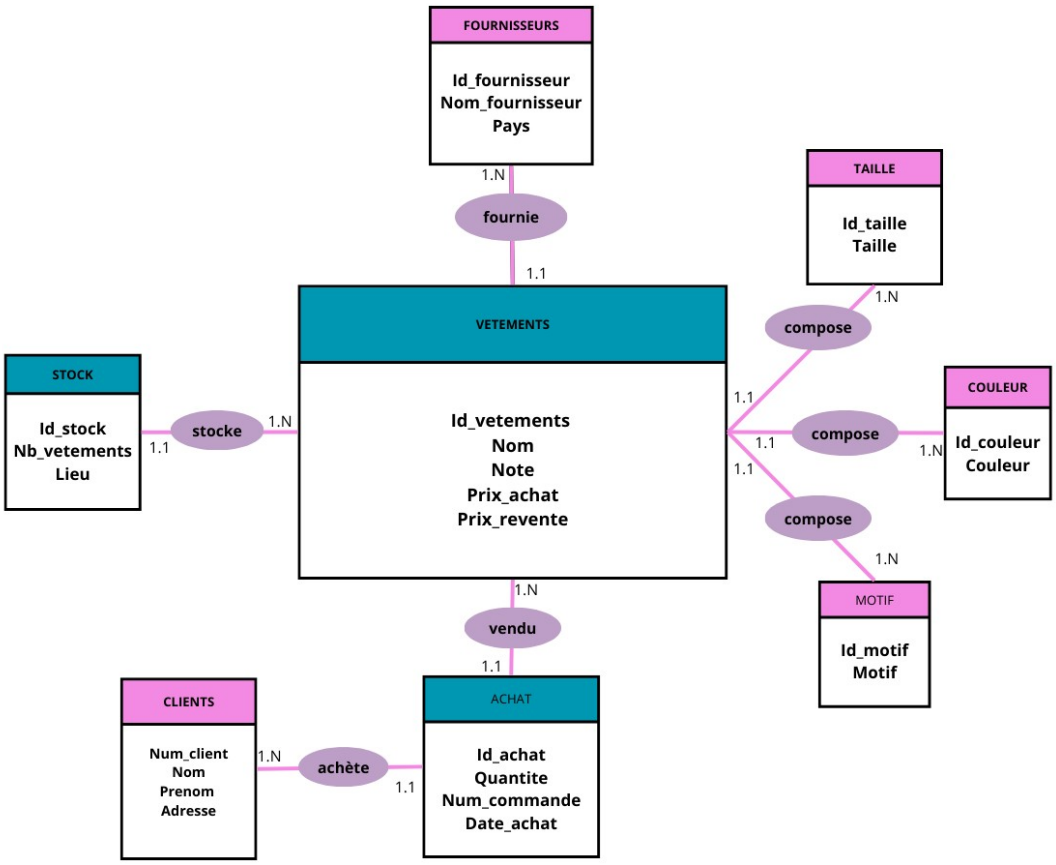
Avec ma collaboratrice Iria De Meyer, nous avons eu l'idée de créer **Lys & Or**, une marque dédiée à la vente de vêtements luxueux, sélectionnés auprès de grands fournisseurs et emblèmes de la mode, dans une région éloignée du luxe.

Nous proposons un catalogue restreint d'articles, afin de garantir une qualité exceptionnelle, une valeur clé de notre enseigne. Nos vêtements offrent une large gamme de tailles, non limitée à une taille unique, ainsi que divers coloris et motifs, permettant à chacun de trouver son bonheur dans l'un de nos deux magasins, régulièrement mis à jour pour garantir des stocks suffisants et accueillir les nouveautés à chaque saison.

Découvrez l'élégance et l'exclusivité de **Lys & Or** à travers l'interface Python que nous avons développée, permettant une gestion simplifiée de notre base de données.

1 - Implémentation de la base de données :

1.1 Modèle entités – associations :



Les tables en rose représentent les tables qui n'utilisent pas de clés étrangères. Et les tables bleues représentent les tables qui en utilisent. Cela est simplement un point de repère pour nous permettre par la suite de réaliser le schéma relationnel en éviter, peut-être, quelques fautes.

On peut constater qu'un motif, une taille et une couleur composent un vêtement. De plus, il est fourni par un fournisseur.
Le stock stocke, comme son nom l'indique, des vêtements.
Un client achète un achat, et donc un ou plusieurs vêtements qui est donc vendu.

1.2 Schéma relationnel :

Voici le schéma relationnel de notre base de données :

FOURNISSEURS(Id_founisseur : INT, Nom_fournisseur : TEXT, Pays : TEXT)

TAILLE(Id_taille : INT, Taille : TEXT)

COULEUR(Id_couleur : INT, Couleur : TEXT)

MOTIF(Id_motif : INT, Motif : TEXT)

VETEMENTS(Id_vetement : INT, Nom : TEXT, Note : INT,
Id_taille : TEXT, # Id_couleur : TEXT, # Id_motif : TEXT Prix_achat : INT,
Prix_revente : INT, # Id_fournisseur : INT)

CLIENTS(Num_client : INT, Nom : TEXT, Prenom : TEXT, Adresse : TEXT)

STOCK(Id_stock : INT, # Id_vetement INT, Nb_vetements : INT, Lieu TEXT)

ACHATS(Id_achat : INT, Quantite : INT, Num_commande : INT, Date_achat : DATE,
Num_client : INT, # Id_vetement : INT)

Dans ce modèle relationnel, la contrainte d'entité est présente. Elle permet de s'assurer que chaque entité de la table est unique, la clé primaire permet de garantir cette contrainte.

Il y a également la contrainte de référence qui créent des associations entre deux relations. Elle permet de préserver l'intégrité d'une base de données, elle vérifie que toutes les valeurs de la clé étrangère correspondent à des valeurs présentes dans la clé primaire.

Enfin, la contrainte de domaine est aussi présente. Elle permet de restreindre les valeurs d'un attribut à un domaine et évitent que l'on puissent saisir des valeurs illégales ou aberrantes.

1.3 Création des tables sur DB Browser :

On commence par créer les tables qui ne possèdent pas de clé étrangère, donc les tables en roses dans le modèle entité-association : la table TAILLE, la table COULEUR, la table MOTIF, la table FOURNISSEURS et la table CLIENTS.

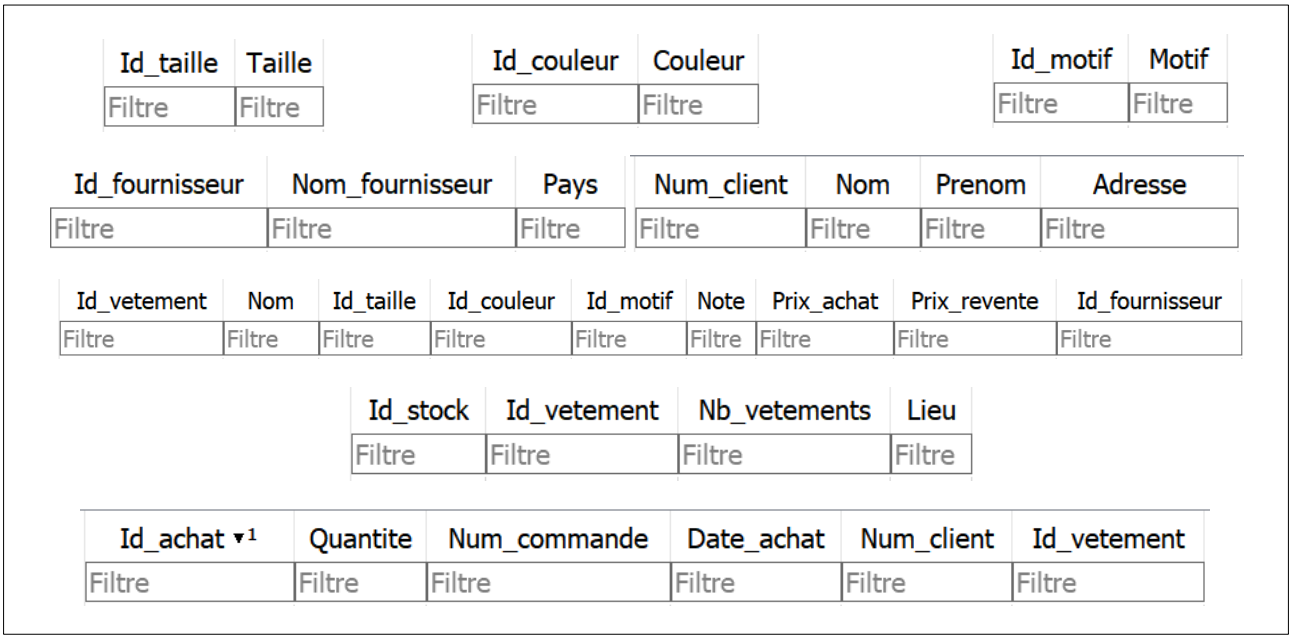
```
CREATE TABLE TAILLE ( Id_taille, Taille ) ;
CREATE TABLE COULEUR ( Id_couleur, Couleur ) ;
CREATE TABLE MOTIF ( Id_motif, Motif ) ;
```

```
CREATE TABLE FOURNISSEURS ( Id_fournisseur, Nom_fournisseur, Pays ) ;
CREATE TABLE CLIENTS ( Num_client, Nom, Prenom, Adresse ) ;
```

Puis on crée les trois tables restantes avec les clés étrangères, donc celles en bleues dans le modèle entité-association : en commençant par la table VETEMENTS, puis la table STOCK et la table ACHATS.

```
CREATE TABLE VETEMENTS ( Id_vetement, Nom, Note, Id_taille, Id_couleur, Id_motif,
                         Prix_achat, Prix_revente, Id_fournisseur )
CREATE TABLE STOCK ( Id_stock, Id_vetement, Nb_vetements, Lieu )
CREATE TABLE ACHATS ( Id_achat, Quantite, Num_commande, Date_achat, Num_client,
                      Id_vetement )
```

On obtient des tables vides, cependant elles seront remplis par la suite avec le programme python. Comme si dessous :



2 - Création de la base de données :

Les requêtes SQL suivantes sont, dans le programme python, toutes insérées dans une fonction complète qui permet de supprimer toutes les erreurs qui pourraientt arriver et renvoie un message pour informer d'un éventuel problème. Pour que les requêtes SQL soient plus compréhensibles, je vais organiser par tables dans chaque parties.

2.1 Les actions sur la BDD :

2.1.1 :

On commence donc avec la table TAILLE dans laquelle, en action, on peut ajouter et supprimer une taille :

```
INSERT INTO TAILLE ( Id_taille, Taille )
VALUES ( ?, ? ) ;

DELETE FROM TAILLE
WHERE Taille = ? ;
```

Les points interrogations, dans la fonction, sont remplacés par des valeurs entrés en argument par l'utilisateur lors de la mise en marche du programme. Mais pour l'exemple en les remplaçants par 2 et « M », on obtient : une table remplit avec une taille avec un Id_taille de 2 et une Taille « M » pour la première requête. Et on obtient la table vide du départ si l'on garde les mêmes valeurs.

Id_taille	Taille
Filtre	Filtre
2	M

Id_taille	Taille
Filtre	Filtre

2.1.2 :

Ensuite, il y a la table COULEUR dans laquelle, en action, on peut ajouter et supprimer une couleur :

```
INSERT INTO COULEUR ( Id_couleur, Couleur )
VALUES ( ?, ? ) ;

DELETE FROM COULEUR
WHERE Couleur = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations par 5 et « rouge », on obtient : une table remplit avec une couleur avec un Id_couleur de 5 et une Couleur « rouge » pour la première requête. Et on obtient la table vide du départ si l'on garde les mêmes valeurs.

Id_couleur	Couleur
Filtre	Filtre
5	rouge

Id_couleur	Couleur
Filtre	Filtre

2.1.3 :

Puis, il y a la table MOTIF, dans laquelle, en action, on peut ajouter et supprimer un motif :

```
INSERT INTO MOTIF ( Id_motif, Motif )
VALUES ( ?, ? ) ;

DELETE FROM MOTIF
WHERE Motif = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations par 2 et « carreau », on obtient : une table remplit avec un motif avec un Id_motif de 2 et une Motif « carreau » pour la première requête. Et on obtient la table vide du départ si l'on garde les mêmes valeurs.

Id_motif	Motif
Filtre	Filtre
2	carreau

Id_motif	Motif
Filtre	Filtre

2.1.4 :

Ensuite, il y a la table FOURNISSEURS dans laquelle, en action, on peut ajouter et supprimer un fournisseur :

```
INSERT INTO FOURNISSEURS ( Id_fournisseur, Nom_fournisseur, Pays )
VALUES ( ?, ?, ? ) ;

DELETE FROM FOURNISSEURS
WHERE Motif = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations par 2, « Chanel » et « Espagne », on obtient : une table remplit avec un fournisseur avec un Id_fournisseur de 2, un Nom_fournisseur « Chanel » et un Pays « Espagne » pour la première requête. Et on obtient la table vide du départ si l'on garde les mêmes valeurs.

Id_fournisseur	Nom_fournisseur	Pays
Filtre	Filtre	Filtre
2	Chanel	Espagne

Id_fournisseur	Nom_fournisseur	Pays
Filtre	Filtre	Filtre

2.1.5 :

Puis, il y a la table CLIENTS, dans laquelle, en action, on peut ajouter et supprimer un client :

```
INSERT INTO CLIENTS ( Num_client, Nom, Prenom, Adresse )
VALUES ( ?, ?, ?, ? ) ;

DELETE FROM CLIENTS
WHERE Num_client = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations par 344, « Delacour », « Marie » et « 34 Avenue Foch », on obtient : une table remplit avec un client avec un Num_client de 344, un Nom « Delacour », un Prenom « Marie » et une Adresse « 34 Avenue Foch » pour la première requête. Et on obtient la table vide du départ si l'on garde les mêmes valeurs.

Num_client	Nom	Prenom	Adresse
Filtre	Filtre	Filtre	Filtre
344	Delacour	Marie	34 Avenue Foch

Num_client	Nom	Prenom	Adresse
Filtre	Filtre	Filtre	Filtre

2.1.6 :

Ensuite, il y a la table VETEMENTS dans laquelle, en action, on peut ajouter et supprimer un vêtement :

```
INSERT INTO VETEMENTS ( Id_vetement, Nom, Id_taille, Id_couleur, Id_motif, Note,
      Prix_achat, Prix_revente, Id_fournisseur )
VALUES ( ?, ?, ?, ?, ?, ?, ?, ?, ?) ;

DELETE FROM VETEMENT
WHERE Nom = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par 1, « Robe », 2, 5, 2, 8, 400, 800, et 2, on obtient : une table remplit avec un vêtement avec un Id_vetement de 1, un Nom « Robe », un Id_taille de 2, un Id_couleur de 5, un Id_motif de 2, une Note de 8, un Prix_achat de 400, un Prix_revente de 800 et un Id_fournisseur de 2 pour la première requête. Et on obtient la table vide du départ si l'on garde les mêmes valeurs.

Id_vetement	Nom	Id_taille	Id_couleur	Id_motif	Note	Prix_achat	Prix_revente	Id_fournisseur
Filtre	Filtre	Filtre	Filtre	Filtre	Filtre	Filtre	Filtre	Filtre
1	Robe	2	5	2	8	400	800	2

Id_vetement	Nom	Id_taille	Id_couleur	Id_motif	Note	Prix_achat	Prix_revente	Id_fournisseur
Filtre	Filtre	Filtre	Filtre	Filtre	Filtre	Filtre	Filtre	Filtre

2.1.7 :

Puis, il y a la table ACHATS, dans laquelle, en action, on peut ajouter et supprimer un achat :

```
INSERT INTO ACHATS ( Id_achat, Quantite, Num_commande, Date_achat, Num_client,
                     Id_vetement )
VALUES ( ?, ?, ?, ?, ?, ? ) ;

DELETE FROM ACHATS
WHERE Id_achat = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par 2, 2, 4678, « 09/12/2024 », 344 et 1, on obtient : une table remplit avec un achat avec un Id_achat de 2, une Quantite de 2, un Num_commande de 4678, une Date_achat du « 09/12/2024 », un Num_client de 344 et un Id_vetement de 1 pour la première requête. Et on obtient la table vide du départ si l'on garde les mêmes valeurs.

Id_achat	Quantite	Num_commande	Date_achat	Num_client	Id_vetement
Filtre	Filtre	Filtre	Filtre	Filtre	Filtre
2	2	4678	09/12/2024	344	1

Id_achat ▾1	Quantite	Num_commande	Date_achat	Num_client	Id_vetement
Filtre	Filtre	Filtre	Filtre	Filtre	Filtre

2.1.8 :

Pour finir, il y a la table STOCK, dans laquelle, en action, on peut ajouter, supprimer un stock, ajouter en plus et enlever une quantité du stock :

```
INSERT INTO STOCK ( Id_stock, Id_vetement, Nb_vetements, Lieu )
VALUES ( ?, ?, ?, ? ) ;

DELETE FROM STOCK
WHERE Id_vetement = ? AND Lieu = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par 5, 4, 58, et « Paris », on obtient : une table remplit avec un stock avec un Id_stock de 5, un Id_vetement de 4, un

Nb_vetements de 58 et un Lieu « Paris » pour la première requête. Et on obtient la table vide du départ si l'on garde les mêmes valeurs.

Id_stock	Id_vetement	Nb_vetements	Lieu
Filtre	Filtre	Filtre	Filtre
5	4	58	Paris

Id_stock	Id_vetement	Nb_vetements	Lieu
Filtre	Filtre	Filtre	Filtre

```
UPDATE STOCK SET Nb_vetements = Nb_vetements + ?
WHERE Id_stock = ? ;

UPDATE STOCK SET Nb_vetements = Nb_vetements - ?
WHERE Id_stock = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par 10 et 4, on obtient : une table remplit avec un stock avec un Id_stock de 5, un Id_vetement de 4, un Nb_vetements de 68 et un Lieu « Paris » pour la première requête. Et on obtient la même table du départ si l'on garde les mêmes valeurs.

Id_stock	Id_vetement	Nb_vetements	Lieu
Filtre	Filtre	Filtre	Filtre
5	4	68	Paris

Id_stock	Id_vetement	Nb_vetements	Lieu
Filtre	Filtre	Filtre	Filtre
5	4	58	Paris

2.2 Les mise à jour de la BDD :

Dans chaque exemple, on part de la base de données rempli avec les exemples précédents.

2.2.1 :

On commence donc avec la table CLIENTS dans laquelle, en mise à jour, on peut modifier le nom du client en cas de mariage ou de divorce, et l'adresse du client en cas de déménagement :

```
UPDATE CLIENTS SET Nom = ?
WHERE Num_client = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par «Marneau» et 344, on obtient : une table modifiée avec un nouveau nom « Marneau ».

```
UPDATE CLIENTS SET Adresse = ?
WHERE Num_client = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par «34 Avenue des champs» et 344, on obtient : une table modifiée avec une nouvelle adresse « 34 Avenue des champs ».

Num_client	Nom	Prenom	Adresse
Filtre	Filtre	Filtre	Filtre
344	Marneau	Marie	34 Avenue des champs

2.2.2 :

Ensuite, il y a la table FOURNISSEURS dans laquelle, en mise à jour, on peut modifier l'origine des produits en cas de délocalisation :

```
UPDATE FOURNISSEURS SET Pays = ?
WHERE Id_fournisseur = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par «France» et 2, on obtient : une table modifiée avec une nouvelle origine « France ».

Id_fournisseur	Nom_fournisseur	Pays
Filtre	Filtre	Filtre
2	Chanel	France

2.2.3 :

Pour finir, il y a la table VETEMENTS dans laquelle, en mise à jour, on peut modifier la taille, la couleur et le motif en cas de nouveaux design, le fournisseur et le prix d'achat en cas de changement, le prix de revente en cas de hausse des prix ou de réduction et la note :

```
UPDATE VETEMENTS SET Id_taille = ?
WHERE Id_vetement = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par 1 et 1, on obtient : une table modifiée avec une nouvelle taille 1.

```
UPDATE VETEMENTS SET Id_couleur = ?
WHERE Id_vetement = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par 3 et 1, on obtient : une table modifiée avec une nouvelle couleur 3.

```
UPDATE VETEMENTS SET Id_motif = ?
WHERE Id_vetement = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par 3 et 1, on obtient : une table modifiée avec un nouveau motif 3.

```
UPDATE VETEMENTS SET Id_fournisseur = ?
WHERE Id_vetement = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par 1 et 1, on obtient : une table modifiée avec un nouveau fournisseur 1.

```
UPDATE VETEMENTS SET Prix_achat = ?
WHERE Id_vetement = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par 300 et 1, on obtient : une table modifiée avec un nouveau prix d'achat de 300.

```
UPDATE VETEMENTS SET Prix_revente = ?
WHERE Id_vetement = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par 500 et 1, on obtient : une table modifiée avec un nouveau prix de revente de 500.

```
UPDATE VETEMENTS SET Note = ?
WHERE Id_vetement = ? ;
```

Pour l'exemple en remplaçant les points d'interrogations respectivement par 7 et 1, on obtient : une table modifiée avec une nouvelle note 7.

Id_vetement	Nom	Id_taille	Id_couleur	Id_motif	Note	Prix_achat	Prix_revente	Id_fournisseur
Filtre	Filtre	Filtre	Filtre	Filtre	Filtre	Filtre	Filtre	Filtre
1	Robe	1	3	3	7	300	500	1

2.3 Analyse des données de la BDD :

2.3.1 :

On commence avec les affichage du contenu de chaque tables afin de connaître toutes les informations relative à chacune.

```
SELECT * FROM TAILLE ;

SELECT * FROM COULEUR ;

SELECT * FROM MOTIF ;

SELECT * FROM FOURNISSEURS ;

SELECT * FROM CLIENTS ;
```

Ces requêtes ci-dessus permettent d'obtenir la totalité des informations sur chaque éléments de la table soit la table en entière.

```
SELECT Id_vetement, Nom, Taille, Couleur, Motif, Note, Prix_revente, Nom_fournisseur
FROM VETEMENTS
JOIN TAILLE ON VETEMENTS.Id_taille = TAILLE.Id_taille
JOIN COULEUR ON VETEMENTS.Id_couleur = COULEUR.Id_couleur
JOIN MOTIF ON VETEMENTS.Id_motif = MOTIF.Id_motif
JOIN FOURNISSEURS ON VETEMENTS.Id_fournisseur = FOURNISSEURS.Id_fournisseur ;
```

Cette requête renvoie toutes les informations de la table VETEMENTS. Cependant à la place de renvoyer les id des clés étrangères, elle renvoie le nom des informations pour permettre une meilleure compréhension.

Id_vetement	Nom	Taille	Couleur	Motif	Note	Prix_revente	Nom_fournisseur
1	Robe	XS	noir	fleur	7	500	Dior
4	Jean	M	rouge	carreau	8	550	Chanel
7	Jean	M	rouge	carreau	8	550	Chanel

```
SELECT Quantite, Num_commande, Date_achat, CLIENTS.Nom, CLIENTS.Prenom,
      VETEMENTS.Nom
FROM ACHATS
JOIN VETEMENTS ON ACHATS.Id_vetement = VETEMENTS.Id_vetement
JOIN CLIENTS ON ACHATS.Num_client = CLIENTS.Num_client ;
```

Cette requête renvoie toutes les informations de la table ACHATS. Cependant à la place de renvoyer les id des clés étrangères, elle renvoie le nom des informations pour permettre une meilleure compréhension.

Quantite	Num_commande	Date_achat	Nom	Prenom	Nom
2	4678	09/12/2024	Marneau	Marie	Robe
2	4678	10/12/2024	Marneau	Marie	Jean

```
SELECT Nom, Nb_vetements, Lieu FROM STOCK
JOIN VETEMENTS ON STOCK.Id_vetement = VETEMENTS.Id_vetement ;
```

Cette requête renvoie toutes les informations de la table ACHATS. Cependant à la place de renvoyer les id des clés étrangères, elle renvoie le nom des informations pour permettre une meilleure compréhension.

Nom	Nb_vetements	Lieu
Jean	68	Paris

2.3.2 :

Ensuite, il y a une requête afin de calculer le montant total d'un achat :

```
SELECT SUM (Quantite * Prix_revente) FROM ACHATS
JOIN VETEMENTS ON ACHATS.Id_vetement = VETEMENTS.Id_vetement
WHERE Num_commande= ? ;
```

Cette requête renvoie le montant total d'un achat en multipliant la quantité commandée par le prix de revente du vêtement puis additionne si il y a plusieurs articles dans la commande. Par exemple en remplaçant le point d'interrogation par le numéro de commande 4678, on obtient :

SUM (Quantite * Prix_revente)
2100

2.3.3 :

Puis, il y a des requêtes afin de calculer les bénéfices :

```
SELECT SUM( Prix_revente - Prix_achat ) * SUM( Quantite ) FROM ACHATS
JOIN VETEMENTS ON ACHATS.Id_vetement = VETEMENTS.Id_vetement
WHERE ACHATS.Id_vetement = ? ;
```

Cette requête renvoie le bénéfice pour un vêtement sur la totalité de ses ventes en soustrayant le prix d'achat au prix de revente puis en le multipliant par le nombre d'articles commandé en tout.

SUM(Prix_revente - Prix_achat) * SUM(Quantite)
400

```
SELECT SUM( ( Prix_revente - Prix_achat ) * Quantite ) FROM ACHATS
JOIN VETEMENTS ON ACHATS.Id_vetement = VETEMENTS.Id_vetement ;
```

Cette requête renvoie le bénéfice pour la totalité des ventes en soustrayant le prix d'achat au prix de revente puis en le multipliant par le nombre d'articles, et enfin en additionnant toutes les lignes obtenue précédemment.

SUM((Prix_revente - Prix_achat) * Quantite)
2200

2.3.4 :

Ensuite, il y a des requêtes afin de calculer les gains :

```
SELECT Prix_revente * SUM( Quantite ) FROM ACHATS
JOIN VETEMENTS ON ACHATS.Id_vetement = VETEMENTS.Id_vetement
WHERE ACHATS.Id_vetement = ? ;
```

Cette requête renvoie le gain pour un vêtement sur la totalité de ses ventes en multipliant le prix de revente par le nombre d'articles commandé en tout.

Prix_revente * SUM(Quantite)
1000

```
SELECT SUM( Prix_revente * Quantite ) FROM ACHATS
JOIN VETEMENTS ON ACHATS.Id_vetement = VETEMENTS.Id_vetement ;
```

Cette requête renvoie le gain pour la totalité des ventes en multipliant le prix de revente par le nombre d'articles, et enfin en additionnant toutes les lignes obtenue précédemment.

SUM(Prix_revente * Quantite)
7600

2.3.5 :

Et pour finir, il y a une requête afin de définir le meilleur vêtement par rapport aux ventes :

```
SELECT Nom, SUM ( Quantite ) AS total_vendu FROM ACHATS
JOIN VETEMENTS ON ACHATS.Id_vetement = VETEMENTS.Id_vetement
GROUP BY VETEMENTS.Nom
ORDER BY total_vendu DESC
LIMIT 1 ;
```

Cette requête renvoie le meilleur vêtement par rapport aux ventes. Le « LIMIT 1 » permet de récupérer seulement le premier élément de la sélection.

Nom	total_vendu
Jean	12

En plus de toutes ces requêtes montrées ci-dessus se trouve dans le programme python des requêtes permettant de vérifier l'existence de certains éléments avant n'importe quelle actions.

3 - Création de l'interface :

3.1 Interface python :

Nous avons décidé de faire une première page d'accueil qui propose trois choix : voir les statistiques de la boutiques, modifier la base de données et quitter l'interface.

Dans le menu statistique, nous pouvons accéder aux gains totaux, aux gains par produit, aux bénéfices totaux, aux bénéfices par produit et au meilleur article.

Dans le menu de modification, nous pouvons accéder aux différentes gestions des tables. Ce qui nous amène sur une nouvelle page. Celle-ci peut, selon la table, ajouter, afficher, supprimer ou modifier un élément.

Voir le programme python joint : programme_entier.

3.2 Interface tkinter :

Nous avons essayé de réaliser un interface tkinter. Cependant, le langage étant inconnu, seul un essai bref a été réalisé afin, seulement, d'essayer. Mais il est possible de le voir à la toute fin de la page de programme python.

4 – Déroulement du projet :

4.1 Organisation :

Au début de notre projet, afin de définir la base de notre travail, nous avons travailler ensemble pour réaliser le modèle relationnel et le schéma entité-association. Sur différentes plateformes en ligne, nous permettant de travailler en même temps sur les documents en voyant en temps réel les modifications de l'autre.

Une fois la base créer, nous avons décidé de nous répartir les tâches. Dans un premier temps nous avions convenu que je devais m'occuper des tables CLIENTS, ACHATS, FOURNISSEURS et STOCK pour leur création sur DB Browser et leur programme python; et Iria devait s'occuper des tables TAILLE, COULEUR, MOTIF, et VETEMENTS pour leur création sur DB Browser et leur programme python. Puis l'interface devait être réaliser par nous deux

Cependant, au fil du temps, nous nous sommes rendu compte qu'il était plus simple pour la création du code que je m'occupe de tout le programme python pour la gestion des tables et Iria s'occupe de la base de données et de l'interface en python et l'essaie de tkinter.

A chaque problème que l'on a rencontré, nous nous sommes tout de même entraidé pour ne pas se laisser bloqué et pouvoir avancé convenablement.

4.2 Choix de développement :

Notre programme pour la gestion de la base de données est composé d'une seule page. La page est constitué de fonctions, un choix que nous avons effectuer afin de simplifier par la suite la programme de l'interface. De plus, cela permet d'éviter le blocage du programme à cause d'une erreur, à la place la fonction renvoie un message pour informer de l'origine du problème.

Pour l'interface, nous avons pris la décision de l'effectuer en python. En effet, pour l'utilisation de notre programme le plus simple pour nous a été de le réaliser en python pur afin de gagner du temps. De plus, nous avons essayé de réaliser un interface tkinter, surtout Iria, seulement c'est un langage inconnu qui nous provoquait beaucoup d'erreur nous avons donc préféré laisser cette idée.

4.3 Les difficultés rencontrées :

Différentes difficultés sont intervenues dans la création de notre projet. Notamment le manque de connaissances pour certaines méthodes ou termes du langage python ou tkinter, nous bloquant sur quelques lignes de code. Cependant, nous avons vite dépassé cette difficulté grâce à différents sites tels que w3school qui nous ont permis d'en apprendre plus et de débloquer le code. Notre code n'étant pas très compliqué, résoudre nos problèmes était assez simple avec cette aide.

Le manque de connexion durant une période des vacances a ralenti la progression, vite rattrapé par la suite.

Durant l'écriture du code, nous voulions au départ effectuer plusieurs pages python afin de le rendre plus lisible néanmoins nous avons rencontré des problèmes de liens entre les pages sans comprendre pourquoi. Alors nous avons fait le choix de regrouper toutes nos fonctions et les interfaces sur la même page python.

Malgré les différentes difficultés, nous avons réussi à les surmonter pour finir notre projet.

4.4 Les limites du projet :

Notre projet est assez complet, il évite l'apparition d'erreurs et gère le problème de clés primaires et clés étrangères. Cependant il comporte également ses limites. En effet, par exemple les clés primaire doivent être inséré par l'utilisateur, et non gérées par les programmes python.

Bien qu'un maximum de vérifications sont faites dans le but d'éviter des entées invalides, il peut rester des cas d'erreurs non gérés.

En supplément, nos fonctions sont souvent longues à cause des vérifications effectuer, il serait préférable de créer un fonction de « vérification » afin de rétrécir les autres.

Pour finir, une fois une commande exécute dans l'interface python, il est nécessaire de relancer le programme pour pouvoir exécuter une nouvelle commande. Ce problème n'a pas pu être réglé à cause du manque de temps malgré les très nombreuses heures de travail sur ce projet.

Un projet quel qu'il soit peut toujours être amélioré.

4.5 Les échecs lors du projet :

Nous avons rencontré plusieurs échecs notamment lors de la création de la base de données et le commencement du programme python. A cause de la conception de la base de données, certaines fonctions étaient totalement impossibles à réaliser pour notre niveau ou trop long, alors nous avons dû repenser la conception, refaire le modèle entité-association et le schéma relationnel à plusieurs reprises jusqu'à trouvé une base de données cohérente et programmable. Cet échec était causé par la présence des éléments taille, couleur et motif dont nous ne savions pas comment les positionner.

Également, comme dit dans les limites du projet, une page « gestion_vérification » aurait été la bienvenue. Néanmoins, nous avons essayé d'en créer une, mais les vérifications à effectuer sont ciblées ce qui complexifie la création de ses fonctions qui doivent être générales pour convenir à un maximum de fonctions de gestion. Au final, nous n'avons pas réussi sa création et avons abandonné cette idée qui aurait pourtant pu permettre une meilleure compréhension du code.

Mais aussi, nous aurions aimez réaliser la gestion des différentes tables à l'aide de la programmation objet, cependant, nous n'avons pas réussi à mettre en place cette idée. Alors, nous avons décidé d'utiliser des fonctions, plus simple à réaliser.

Et pour finir, nous avons essayer de créer un interface tkinter. Cependant cela ne reste qu'un simple essaie car nous n'avons pas réussir à réaliser quelque chose de concret. De plus, nous choisi de plus consacrer de temps au reste du projet, que nous maîtrisons, plutôt qu'à un langage encore inconnu.

5 – Conclusion :

Au final, ce projet constitue une expérience enrichissante qui nous a permis de mettre en pratique nos compétences en programmation python et SQL et d'en apprendre plus sur le langage tkinter. Malgré les difficultés et échecs rencontré, nous avons su passer au dessus pour à la fin réaliser un projet complet, mettant nos idées de base en fonctionnement.